

Appendix B: Tag-length-value (TLV) Schema Definitions

B.1. Introduction

A TLV Schema provides a simple textual description of the structure of data encoded in the [Matter TLV](#) format. A single TLV Schema MAY define the structure of multiple different TLV-encoded payloads. This section describes the syntax one can use to define a TLV Schema.

B.1.1. Basic Structure

A TLV Schema takes the form of a series of definitions. Each definition describes some construct, such as a data type. Each definition has an associated human readable name separated from the definition with a $\Rightarrow$ symbol. As a mnemonic device, it is useful to read the $\Rightarrow$ symbol as “is a”. For example, the following definition defines a data type that MAY be used to represent a sensor sample:

Example

```
/** Sensor sample structure */
sensor-sample => STRUCTURE
{
    timestamp [1] : UNSIGNED INTEGER [ range 32-bits ],
    value [2] : FLOAT64,
}
```

This example would be read as “*sensor-sample* is a structure containing a *timestamp* and *value*”.

A TLV Schema MAY contain multiple definitions. The order of definitions within a TLV Schema is unimportant.

B.1.2. Keywords

TLV Schemas employ various keywords when describing a construct. These keywords (e.g. **STRUCTURE**, **SIGNED INTEGER**, and **range**) are an inherent part of the schema language. Keywords in TLV Schemas are always case-insensitive. However, by convention, keywords associated with types and other high-level constructs are capitalized for emphasis in text-only contexts.

B.1.3. Naming

Each definition in a TLV Schema assigns a human-readable name to the construct being defined. This name serves both as a descriptive title as well as a means to refer to the construct from elsewhere in the schema.

Names in TLV Schemas are limited to ASCII alphanumeric characters, plus dash (-) and underscore (_). Additionally, all names SHALL begin with either an alphabetic character or an underscore. In